

Tipo 1: HTML + JS + CSS integrando la base de datos mediante **PHP**

Requisitos obligatorios:

1. Estructura de la interfaz de usuario (UI):

- Incluye al menos 4 páginas diferentes de tu sitio web. Entre ellas tendrás que incluir la página principal de tu aplicación, que es obligatoria, y otras páginas adicionales elegidas según las funcionalidades implementadas. (Por ejemplo, si tienes desarrollado un sistema de autenticación, una de las páginas adicionales podría ser la pantalla de login)
- Emplea etiquetas semánticas apropiadas para mejorar la accesibilidad de las páginas(header, nav, main, aside, footer), organiza el css de forma modular, emplea variables, clases reutilizables, etc..
- Contempla el uso de media queries o técnicas de diseño responsivo para adaptar tu sitio web a distintos dispositivos (móviles, tablets y escritorio)

2. PHP/MariaDB

- Al menos 3 entidades en la BD con una relación N:M y 1:N.
- Implementar autenticación básica con sesiones en PHP.
- Uso de formularios con validación (lado cliente y servidor).
- La implementación debe de evitar ataques de inyección SQL y similares.
- Los formularios sensibles (login, alta, edición, borrado) deben incluir tokens CSRF y verificarlos en el servidor.
- El SQL y trazas no deben exponerse al usuario final. Loguear errores de servidor de forma segura
- Password hashing con password_hash()/password_verify().

Tipo 2: HTML + JS + CSS consumiendo un **API REST** de elaboración propia desde JavaScript.

Requisitos obligatorios:

1. Estructura de la interfaz de usuario (UI):

- Incluye al menos 4 páginas diferentes de tu sitio web. Entre ellas tendrás que incluir una página principal
- Emplea etiquetas semánticas apropiadas para la construcción de las páginas (header, nav, main, aside, footer).

- Contempla el uso de **media queries** o técnicas de diseño responsivo para adaptar tu sitio web a distintos dispositivos.

2. Django (backend):

- Exponer, como mínimo, 5 endpoints, que deben ser un GET, un POST, un PUT, un PATCH y un DELETE. Se pueden implementar finalmente tantos endpoints como se quiera (predeciblemente, más que 5).
- Se usará al menos una vez, en el total de la fachada REST, cada uno de los siguientes mecanismos de recepción de parámetros. El diseño de los parámetros debe ser semánticamente coherente.
 1. Parámetros en la ruta de la URL (path params)
 2. Parámetros en la query de la URL (query params)
 3. Parámetros en el cuerpo de la petición
- Se usará JSON como formato de intercambio de datos en cuerpo y respuesta de la petición.
- Existirán al menos tres modelos que se usen en la fachada REST, con al menos una relación N:N ó ternaria presente en el diseño de los mismos.

Tipo 3: React + CSS consumiendo un **API REST de elaboración propia.**

Requisitos obligatorios:

1. React (frontend)

- Emplear la librería react-router-dom y recoger al menos 4 páginas diferentes.
- Enviar peticiones asíncronas de tipo GET, POST, PUT y DELETE como vehículos de la funcionalidad de la web en aquellas páginas donde sea coherente
- Tener un nivel razonable de complejidad en cuanto al número de componentes y relaciones entre ellos. Por ejemplo: No es válido que sólo exista un componente para cada página. Aquellas páginas más elaboradas tendrán, congruentemente, un número de componentes que encapsule razonablemente las distintas piezas de la interfaz.

2. Django (backend):

- Exponer, como mínimo, 4 endpoints, que deben ser un GET, un POST, un PUT y un DELETE. Se pueden implementar finalmente tantos endpoints como se quiera (predeciblemente, más que 4).
- Se usará al menos una vez, en el total de la fachada REST, cada uno de los siguientes mecanismos de recepción de parámetros. El diseño de los parámetros debe ser semánticamente coherente.
 1. Parámetros en la ruta de la URL (path params)
 2. Parámetros en la query de la URL (query params)

3. Parámetros en el cuerpo de la petición

- o Se usará JSON como formato de intercambio de datos en cuerpo y respuesta de la petición.
- o Existirán al menos tres modelos que se usen en la fachada REST, con al menos una relación N:N ó ternaria presente en el diseño de los mismos.